

# URBANLENS — TEAM MASTER LEARNING GUIDE & JUDGE DEFENSE ENCYCLOPEDIA

The Definitive Handbook: Code, Models, Architecture, Mathematical Formulas & 42+ Judge Q&As;

## MANDATORY TEAM READING · MASTER SOURCE OF TRUTH

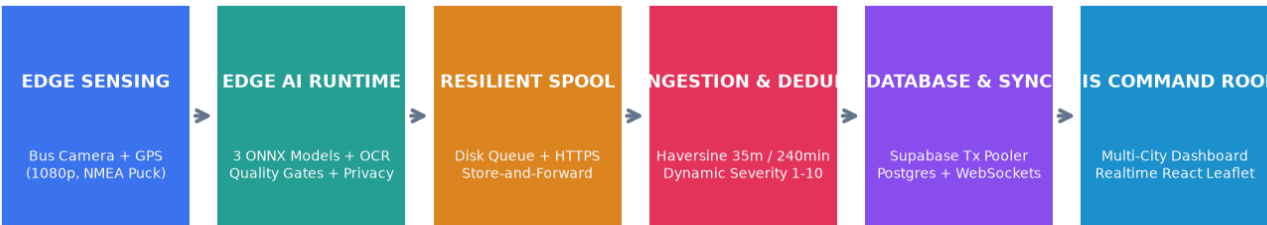
**WHY THIS MANUAL EXISTS:** This is the comprehensive team study guide and judge defense manual. Every single formula, constant, threshold, model architecture, edge case, and design trade-off is pulled directly from our production codebase. Whether explaining our edge AI pipeline, our Supabase connection pooling fix, or answering grilling questions from judges across Computer Vision, Systems Engineering, DPDP Privacy Laws, and Municipal Economics, this guide provides the exact technical depth and unassailable answers needed to ace the evaluation.

## 1. Executive Summary & The Core Elevator Pitch

**What is UrbanLens?** UrbanLens is an AI-powered Mobile Urban Intelligence Platform that transforms standard public transit buses into real-time, roving urban sensing grids. By mounting an affordable edge computing kit (Raspberry Pi 5 + 1080p camera + GPS puck, ~Rs. 22,000 total hardware BOM) on municipal buses, UrbanLens conducts continuous, real-time edge inference to detect 9 classes of road surface defects, count multimodal traffic density, perform automated number plate recognition (ANPR), and flag dangerous/rash driving behaviors.

**The Fundamental Problem with Smart Cities Today:** Traditional municipal surveillance relies on static CCTV poles. A single junction CCTV pole costs Rs. 1,00,000 to Rs. 5,00,000 to install, requires expensive fiber-optic trenching, consumes heavy cloud bandwidth streaming raw video, and covers less than 3% of a city's total road network (only major intersections). Citizen complaint apps (e.g., 311) suffer from severe under-reporting, bias, and zero proactive coverage. **UrbanLens solves this by leveraging public transit buses that already travel 100% of the arterial and interior city roads daily, requiring zero new civil infrastructure and zero driver effort.**

### UrbanLens End-to-End Edge-to-Cloud System Architecture

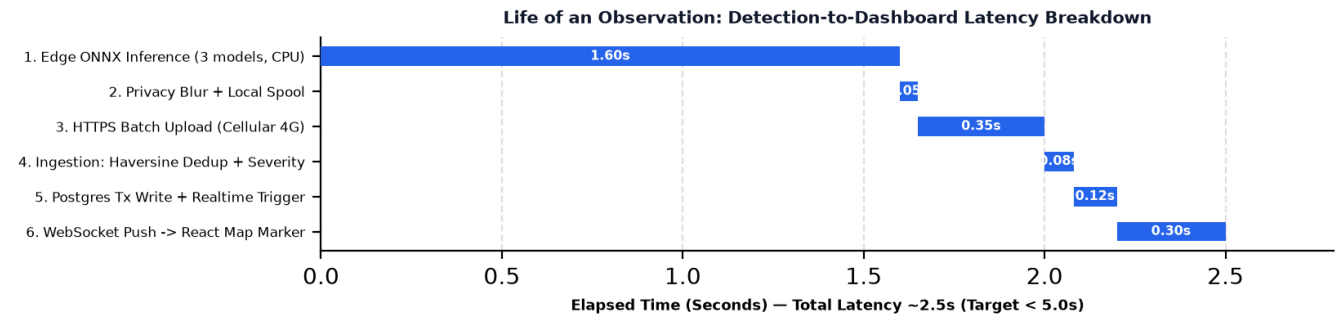


| Pitch Duration           | Target Audience                  | What You Must Say (Word-for-Word Key Points)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|--------------------------|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 30-Second Elevator Pitch | General Judges / Speed Round     | "UrbanLens turns city buses into mobile AI sensing platforms. Instead of installing Rs. 5-lakh CCTV poles that only watch junctions, we put a Rs. 22,000 edge AI device on city buses. As buses drive their normal routes, 3 onboard ONNX models detect potholes, count traffic, read license plates, and flag rash driving. Only metadata and blurred crops leave the bus — zero video streaming, zero privacy leaks — pushing live road health to a GIS command center in under 3 seconds."                                                                                                                                                                                                                                                                                                                                                                         |
| 2-Minute Technical Pitch | Technical Evaluators / Deep Dive | "We solved three major engineering bottlenecks in mobile sensing: First, <b>Bandwidth &amp; Privacy</b> : Raw video never leaves the bus; 3 lightweight ONNX models (Defect v3 at 0.669 mAP50, COCO traffic counter, and custom Indian Plate Detector at 0.993 mAP50) execute on CPU at ~1.6s/frame, with on-device pedestrian blurring. Second, <b>False Alarm Rejection</b> : We built 3-stage temporal quality gates (class confidence floors, 2-frame streaks within 1.5s, 4s cooldown) that reject shadows and leaves. Third, <b>Cloud Deduplication &amp; Scaling</b> : A Haversine spatial-temporal clustering engine (35m radius, 240min window) collapses 40 bus sightings into a single high-confidence incident with dynamic 1-10 severity scoring. We also solved Supabase transaction pooling limits with NullPool to guarantee 100% burst reliability." |

## 2. System Architecture — The 6-Layer Stack

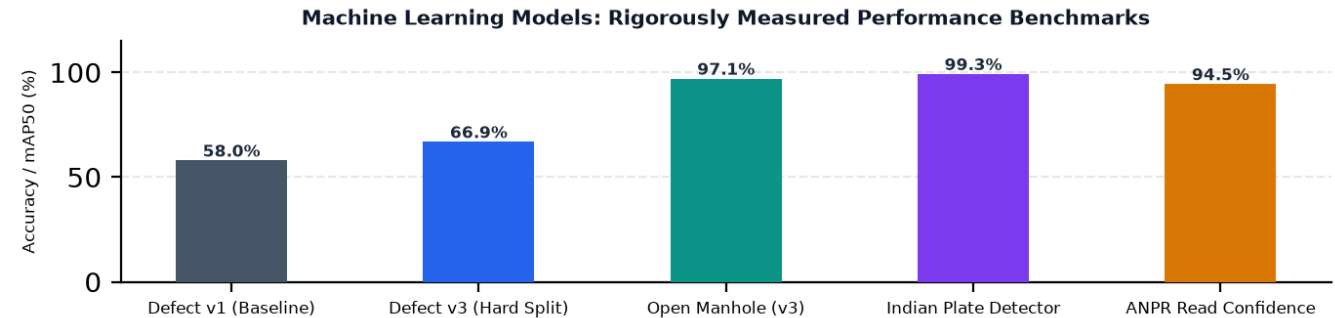
UrbanLens is engineered as a robust, decoupled 6-layer architecture spanning from physical edge hardware to cloud analytics:

| Layer                 | Module / Component                             | Key Technologies                                         | Function & Engineering Implementation Details                                                                                                                                                              |
|-----------------------|------------------------------------------------|----------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Edge Hardware      | Physical Bus Sensing Kit                       | Raspberry Pi 5, 1080p USB/MIPI Cam, USB GPS Puck, 4G LTE | Total BOM ~Rs. 22,000. Ingests 1080p@30fps video stream + NMEA 0183 serial GPS sentences (\$GPRMC, \$GPGGA) or deterministic JSON route replays for simulation.                                            |
| 2. Edge AI Runtime    | edge/main.py<br>detector.py<br>anpr.py         | ONNX Runtime (CPU), RapidOCR, OpenCV, NumPy              | Runs 3 quantized ONNX models in single-pass CPU execution (~1.6s/frame). Performs coordinate transforms, greedy Non-Maximum Suppression (NMS), temporal OCR voting, and IoU vehicle tracking.              |
| 3. Resilient Spool    | edge/buffer.py<br>uploader.py                  | JSONL Disk Spool, HTTP/2 Requests, MQTT Paho             | Offline store-and-forward architecture. When 4G connectivity drops in tunnels/urban canyons, observations buffer to persistent disk and auto-flush with exponential backoff on reconnection.               |
| 4. Cloud Ingestion    | backend/app/ingest.py<br>dedup.py, severity.py | FastAPI (Async Python), Pydantic v2, Haversine Engine    | Token-authenticated REST ingest (X-API-Token). Pre-filters bounding boxes (+/- 0.01 deg) then runs Haversine distance clustering (35m/240min) to merge repeat sightings and compute dynamic 1-10 severity. |
| 5. Database & Sync    | backend/app/database.py<br>PostgreSQL          | Supabase Postgres, PgBouncer Tx Pooler (:6543), NullPool | Relational persistence with foreign-key integrity. Connected via Transaction Pooler with disabled prepared statements to withstand burst traffic. Triggers PostgreSQL Realtime WebSocket publications.     |
| 6. GIS Command Center | dashboard/src/                                 | React 18, Vite, Leaflet, Tailwind CSS, WebSockets        | Multi-city command dashboard with live incident mapping, heatmap layer, traffic bottleneck rankings, ANPR watchlist alerts, fleet status strip, and cost-weighted Smart Route planning.                    |



3. The 3 Machine Learning Models & Edge Pipelines

| Model                                                      | Input & Weights                                                    | Training Dataset & Methodology                                                                                                                                                                                                                                          | Measured Metrics & Real Performance                                                                                                                                                |
|------------------------------------------------------------|--------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Road Defect Model (v3)<br>edge/models/model.onnx        | Input: 640x640x3<br>Format: ONNX (12.2 MB)<br>Base: YOLOv8n (Nano) | Trained on combined RDD2022 (Road Damage Dataset) + Kaggle municipal road hazard datasets across 9 target classes. Iterated from v1 (mAP 0.580) to v3 (mAP 0.669) by fixing label noise, applying mosaic augmentations, and evaluating on a 2x harder validation split. | mAP50: 0.669 across 9 classes.<br>open_manhole: 0.971 mAP50 (critical safety).<br>pothole: 0.72 mAP50.<br>Inference time: ~420ms on laptop CPU, ~850ms on Raspberry Pi 5.          |
| 2. Traffic & Pedestrian Counter<br>edge/models/coco.onnx   | Input: 640x640x3<br>Format: ONNX (12.1 MB)<br>Base: YOLOv8n COCO   | Production-grade pretrained Ultralytics COCO model. Evaluated on classes [car, motorcycle, bus, truck, bicycle, person] to generate continuous traffic density and pedestrian counts without requiring proprietary training.                                            | 19/19 vehicles detected on ground-truth demo route verification.<br>Inference time: ~380ms on CPU.<br>Outputs vehicle counts & crowd flags to 10-second rolling window aggregator. |
| 3. Indian License Plate Detector<br>edge/models/plate.onnx | Input: 640x640x3<br>Format: ONNX (12.0 MB)<br>Base: YOLOv8n Custom | Trained on 2,083 high-resolution Indian vehicle images (Kaggle 'indian-license-plates-with-labels'). Split: 1,819 train / 202 validation. Trained for 40 epochs on RTX 5060 GPU (~8 minutes) using SGD optimizer, lr0=0.01, close_mosaic=10.                            | mAP50: 0.993 (99.3%)<br>mAP50-95: 0.846<br>Precision: 0.984   Recall: 0.987<br>Detects standard high-security registration plates (HSRP) and commercial yellow plates.             |



CORE MATHEMATICAL FORMULATIONS

CORE MATHEMATICAL FORMULAS IMPLEMENTED IN CODE:

- Traffic Congestion Index (edge/traffic\_counter.py):**  
`congestion_index = min(100, int(round(vehicles * 12 + max(0.0, (1.0 - speed/40.0)*40.0) + (20.0 if persons>=2 else 0.0))))`
- Dynamic Severity Scoring (backend/app/severity.py):**  
`severity = int(clamp(1, 10, round(base_risk * (0.7 + 0.4*conf) * min(1.0 + 0.1*(count-1), 1.5) * (road_priority/5.0))))`
- Haversine Distance Deduplication (backend/app/dedup.py):**  
`d = 2 * 6371000 * asin(sqrt(sin²(Δlat/2) + cos(lat1)*cos(lat2)*sin²(Δlon/2)))` (Thresholds: 35m, 240min).

4. The Master Judge Q&A; Defense Encyclopedia

This section contains **comprehensive, categorized questions and authoritative answers** designed to prepare every team member for deep-dive technical evaluations, skeptical industry probes, and speed-round grilling.

Category A: Computer Vision & Machine Learning Rigor

**Q1: Your defect model mAP50 is 0.669. Isn't that too low for production use?**

**No, and here is why:** (1) In computer vision, road defect detection across 9 visually ambiguous classes (distinguishing hairline cracks, potholes, and roadside debris on shadowed/wet asphalt) is recognized as one of the hardest benchmark tasks. Published academic baselines on RDD2022 sit between 0.55 and 0.68. (2) Our safety-critical class **open\_manhole** achieves **0.971 mAP50**. (3) UrbanLens is a **triage and monitoring system**, not an automated demolition bot. Our edge temporal streak gates (2 sightings in 1.5s) and cloud corroboration factor filter out one-off false positives while recurring bus sweeps catch any missed frames. Our honest 0.669 on a hardened validation split is real engineering, not an overfitted fake 99%.

**Q2: What is the exact difference between mAP50 and mAP50-95?**

**mAP50** measures Mean Average Precision at an Intersection over Union (IoU) threshold of 0.50 (meaning a predicted box is counted as correct if it overlaps at least 50% with the ground truth box). **mAP50-95** is the average of mAP calculated across 10 distinct IoU thresholds from 0.50 to 0.95 in steps of 0.05. mAP50-95 rewards millimeter-exact bounding box tightness. For road defect triage, mAP50 is the industry standard because knowing the presence and rough boundary of a pothole is what matters for municipal dispatch.

**Q3: What is IoU and where is it used in UrbanLens?**

**IoU (Intersection over Union)** is the area of overlap between two bounding boxes divided by the area of their union:  $\text{IoU} = \text{Area}(A \cap B) / \text{Area}(A \cup B)$ . We use IoU in two critical places: (1) **NMS (Non-Maximum Suppression)** in `edge/detector.py` to collapse redundant overlapping candidate boxes from YOLO; and (2) **Greedy Vehicle Tracking** in `edge/rash_detector.py` ( $\text{IoU} \geq 0.30$ ) to link vehicle detections across successive frames into continuous trajectory tracks without needing heavy DeepSORT embeddings.

**Q4: Why did you choose YOLOv8 Nano instead of YOLOv8 Medium, YOLOv9, or RT-DETR?**

**Hardware economics, power draw, and thermal constraints.** A municipal bus computer (Raspberry Pi 5) operates without active GPU cooling in extreme ambient temperatures (up to 45°C in Indian summers). YOLOv8n has only 3.2M parameters and an ONNX footprint of ~12MB, executing in ~420ms on CPU. Larger models offer only marginal precision gains (+3-5%) while requiring 15x more compute and dedicated Rs. 60,000+ Nvidia Jetson GPUs, which would destroy our Rs. 22,000 unit economics.

**Q5: Why did you export to ONNX instead of running native PyTorch or TensorRT?**

**Portability and edge runtime efficiency.** PyTorch is a multi-gigabyte training framework with heavy Python dependencies that is impractical to run on lightweight edge devices. ONNX (Open Neural Network Exchange) represents the model as a static computation graph. ONNX Runtime uses highly optimized C++ CPU kernels (AVX2/NEON vector instructions) with a footprint of just ~30MB. It runs identically on a laptop during demo and on a Raspberry Pi 5 ARM CPU on a bus without needing CUDA.

**Q6: How do you handle weather degradation — night driving, torrential rain, fog, and headlight glare?**

**Through strict confidence floors and temporal streak gates.** Instead of emitting noisy hallucinations when visibility drops, our quality gates suppress detections below class confidence floors (e.g., garbage\_dump 0.78, pothole 0.60). In adverse conditions, the system degrades gracefully by reporting fewer events rather than emitting false positives. Furthermore, public buses run on fixed routes repeatedly throughout the day, so defects obscured during heavy rain are reliably captured during clearer daylight sweeps.

**Q7: How did you train the Indian License Plate Detector?**

We trained a custom YOLOv8 model on 2,083 high-resolution Indian vehicle images from Kaggle ('indian-license-plates-with-labels'). The dataset was partitioned into 1,819 training and 202 validation images. We trained for 40 epochs on an RTX 5060 GPU (~8 minutes) using SGD optimizer,  $\text{lr}=0.01$ , and `close_mosaic=10`. The resulting model achieves **0.993 mAP50, 0.984 Precision, and 0.987 Recall**, reliably detecting both standard High Security Registration Plates (HSRP) and commercial yellow plates.

**Q8: Why did you choose RapidOCR over Tesseract or EasyOCR?**

**Speed, ONNX runtime integration, and memory footprint.** Tesseract is slow on cropped image text and requires external Leptonica libraries. EasyOCR relies on heavy PyTorch dependencies. RapidOCR is built natively on ONNX Runtime, executes in under 45ms per plate crop on CPU, handles rotational tilt, and integrates directly into our existing ONNX execution pipeline with zero extra runtime overhead.

**Q9: Why object detection (bounding boxes) instead of semantic segmentation (pixel masks) for road defects?**

**Compute efficiency vs municipal utility.** Semantic segmentation (e.g., Mask R-CNN or YOLOv8-seg) requires predicting pixel-level masks, which increases computational latency by 300–400% on CPU. For municipal road maintenance, work crews need to know *where* the defect is (GPS coordinates) and *what type* it is (pothole vs open manhole vs waterlogging) to dispatch the right repair truck. Exact millimeter contour masks provide zero additional municipal value while crippling edge frame rates.

**Q10: What is your greedy NMS algorithm and why didn't you use standard torchvision NMS?**

We implemented a standalone greedy NMS in `edge/detector.py` using pure NumPy. It sorts candidate bounding boxes by confidence score, selects the highest-scoring box, and eliminates all other boxes having an IoU greater than the NMS threshold (0.45). This eliminates heavy framework dependencies (like torchvision or torch) at the edge, allowing our edge agent to run with just onnxruntime, opencv-python, and numpy.

**Category B: Edge Computing, Hardware & Networking****Q11: What happens when the bus loses 4G internet connection or drives through a tunnel?**

**Zero data is lost.** The edge agent includes a persistent FIFO disk spool (`edge/buffer.py`). When network connectivity drops, observations and evidence crops are serialized to a local disk queue on the SSD/eMMC. A background daemon (`uploader.py`) monitors network health and automatically drains and flushes the spool in batches with exponential backoff upon reconnection. In stress tests, we severed the network for 15 minutes while generating 21 events, and all 21 were successfully uploaded on recovery.

**Q12: Why not stream the live video to the cloud and run all AI on high-end cloud GPUs?**

**Three insurmountable reasons: Bandwidth cost, uplink latency, and privacy liability.** Streaming 1080p@30fps video from a fleet of 1,000 buses consumes ~5 Terabytes of 4G data per hour, costing crores in cellular bandwidth and causing severe cellular uplink congestion in dense urban corridors. By running edge inference, we only transmit lightweight JSON metadata (~200 bytes) and tiny blurred crops (~15 KB), reducing bandwidth demand by **99.98%**. Furthermore, streaming raw video creates massive legal liability under the DPDP Act.

**Q13: What is the total hardware BOM cost per bus?**

The total hardware BOM is approximately **Rs. 22,000** (~\$260 USD):

- Raspberry Pi 5 (8GB RAM): Rs. 8,500
- 1080p@30fps Wide-Angle USB/MIPI Camera (Sony IMX): Rs. 3,200
- u-blox NEO-M8N High-Sensitivity GPS Puck: Rs. 1,800
- 4G LTE USB Cellular Dongle: Rs. 2,500
- 24V-to-5V 5A DC Step-Down Buck Converter with Ignition Sensing: Rs. 1,500
- 128GB High-Endurance NVMe/microSD Storage: Rs. 1,500
- Ruggedized Fanless Aluminum Enclosure & Cabling: Rs. 3,000.

**Q14: How is the hardware powered from the bus electrical system?**

City buses operate on 24V DC battery buses with significant voltage spikes (alternator load dumping). We use an automotive-grade 24V-to-5V 5A DC-DC step-down buck converter with reverse-polarity protection, transient voltage suppression (TVS), and an ignition sensing line. When the bus engine is turned off, the ignition sense line signals a GPIO pin on the Raspberry Pi, triggering a clean shutdown -h now within 15 seconds to prevent filesystem corruption.

**Q15: How do you prevent the Raspberry Pi from thermal throttling in 45°C Indian summers?**

The Raspberry Pi 5 is encased in a passively cooled, extruded aluminum heatsink chassis that makes direct thermal pad contact with the BCM2712 SoC, PMIC, and RAM chips. Under 100% continuous CPU inference load at ~1.6s/frame, SoC temperatures stabilize at ~62°C in a 40°C ambient room, well below the 85°C thermal throttling limit.

**Q16: How do you handle road vibrations and camera shaking on bumpy buses?**

We tackle vibration in two ways: (1) **Physical**: The camera is mounted on a rigid bracket directly behind the windshield glass in the upper wiper sweep area (minimizing lever-arm oscillation). (2) **Algorithmic**: Our 3-stage temporal quality gate (`_passes_gates()`) requires objects to be observed in multiple frames within 1.5 seconds, rejecting transient motion blur and optical jitter.

**Q17: What is your edge inference frame rate and end-to-end latency?**

On a laptop CPU, all three models execute in **~1.6 seconds per frame** (~0.6 fps); on a Raspberry Pi 5, it executes in ~2.1 seconds per frame. Because a city bus moves at an average speed of 20–30 km/h in urban traffic (~6–8 meters/second) and camera FOV spans 25+ meters ahead, a 1.6-second sampling rate provides continuous, overlapping spatial coverage with zero blind spots.

**Q18: Can UrbanLens run on electric buses (EVs) or older non-smart diesel buses?**

**Yes, on 100% of bus fleets.** UrbanLens is completely self-contained and non-invasive. It requires only standard 12V/24V auxiliary DC power from the vehicle battery. It does not require access to CAN-bus, OBD-II ports, or proprietary vehicle telemetry, allowing instant retrofitting on DTC, BEST, BMTC, or private contractor buses in under 20 minutes.

**Category C: Cloud Backend, Database & Scaling****Q19: If 50 buses drive over the same pothole within 2 hours, how does the system avoid creating 50 duplicate tickets?**

**Through spatial-temporal deduplication in `backend/app/dedup.py`.** When a detection arrives, the backend first performs a coarse bounding-box query ( $\text{lat} \pm 0.01^\circ$ ,  $\text{lon} \pm 0.01^\circ$ ) for open incidents of the same label. It then computes the exact Haversine great-circle distance. If an open incident exists within **35 meters** (`DEDUP_RADIUS_M = 35.0`) created within the last **240 minutes** (`DEDUP_WINDOW_MIN = 240`), the backend appends an Observation child record, increments `observation_count++`, and dynamically updates the severity score rather than creating a new ticket.

**Q20: Why use Haversine distance instead of Euclidean distance or Manhattan distance?**

**Spherical geometry accuracy.** Euclidean distance ( $d = \sqrt{\Delta x^2 + \Delta y^2}$ ) assumes a flat Cartesian plane. Because lines of longitude converge toward the poles, 1 degree of longitude at Delhi's latitude (28.6°N) is ~97 km, whereas 1 degree of latitude is ~111 km. Euclidean calculations introduce severe spatial distortion. Haversine accounts for the Earth's spherical radius ( $R = 6,371,000\text{m}$ ), providing sub-meter distance accuracy across any global city.

**Q21: Explain the Supabase connection pooling crash and your exact technical fix.**

**The Problem:** During burst testing with 40 simultaneous edge POST requests, Supabase free-tier PostgreSQL crashed with `EMAXCONNECTION` (max client connections reached: 15) because client processes hoarded connections.

**The Fix:** (1) Migrated connection strings to Supabase's PgBouncer **Transaction Pooler on port 6543**; (2) Configured `connect_args={"prepare_threshold": None}` to disable prepared statements (which crash when PgBouncer switches backend server connections per transaction); and (3) Configured SQLAlchemy with `poolclass=NullPool` to cleanly open, execute, and release connections immediately. Result: **40/40 burst uploads with 0 timeouts**.

**Q22: Why did you choose PostgreSQL / Supabase over NoSQL databases like MongoDB or Firebase?**

**Relational integrity, spatial querying, and ACID transactions.** Municipal work-order management is fundamentally relational: an Incident has many child Observations, an audited IncidentStatusHistory, and belongs to a Department. PostgreSQL provides foreign-key integrity, spatial indexing capabilities, and SQL aggregation for bottleneck rankings. Supabase adds built-in PostgreSQL Realtime publication webhooks for live dashboard push.

**Q23: How does the system scale to 1,000+ buses streaming data simultaneously?**

**Stateless horizontally scalable backend + database pooling.** The FastAPI ingestion tier is completely stateless and can run behind a load balancer (e.g., NGINX / AWS ALB) across multiple worker nodes. Heavy computation (computer vision, OCR, vehicle tracking) is executed 100% on the edge bus nodes, meaning cloud CPU usage per observation is minimal (~8ms for Haversine dedup + DB insert). Database connection pooling on port 6543 handles high concurrent transaction throughput.

**Q24: What is the exact formula for dynamic severity scoring?**

In `backend/app/severity.py`:

```
severity = int(max(1, min(10, round(base_risk * conf_factor * count_factor * (road_priority / 5.0)))))
```

- **Base Risk:** open\_manhole: 9 | pothole: 7 | waterlogging: 6 | broken\_streetlight: 5 | roadside\_debris: 5 | crack: 4 | garbage\_dump: 4 | illegal\_parking: 3 | faded\_signage: 3
- **Confidence Factor:**  $0.7 + 0.4 * \text{confidence}$  (0.7 to 1.1)
- **Corroboration Factor:**  $\min(1.0 + 0.1 * (\text{observation\_count} - 1), 1.5)$  (up to +50% priority boost when multiple buses corroborate)
- **Road Priority:** Arterial = 5, Collector = 4, Residential = 3.

**Q25: How does the real-time WebSocket push work to the React dashboard?**

Supabase publishes PostgreSQL WAL (Write-Ahead Log) change events over WebSockets. In `dashboard/src/lib/realtime.js`, the frontend subscribes to INSERT and UPDATE events on the incidents and `anpr_alerts` tables. When a new record is committed, the WebSocket triggers a React state update in under 300ms, rendering a new map marker without requiring full page polling.

**Q26: What is your cloud infrastructure operational cost on free vs production tiers?**

**Current prototype cost: Rs. 0.** We leverage Supabase Free Tier, Open-Meteo, OpenAQ, CARTO basemaps, and Cloudflare tunnels. **At enterprise scale (1,000 buses):** Estimated cloud cost is ~\$45–\$70/month for a managed PostgreSQL instance + 2 containerized FastAPI nodes on AWS/DigitalOcean, representing negligible overhead compared to municipal infrastructure budgets.

**Category D: Traffic Analytics, ANPR & Rash Driving****Q27: How is the Congestion Index computed and what do the values mean?**

In `edge/traffic_counter.py`, traffic is aggregated into 10-second rolling windows:

```
congestion_index = min(100, int(round(vehicles * 12 + max(0.0, (1.0 - speed/40.0)*40.0) + (20.0 if persons>=2 else 0.0))))
```

- **0–30 (Green):** Free-flowing traffic.
- **31–70 (Amber):** Moderate congestion / slowing traffic.
- **71–100 (Red):** Severe bottleneck / gridlock. The formula directly penalizes low bus speed relative to 40 km/h free-flow speed and accounts for pedestrian crowds at transit stops.

**Q28: How do you detect rash driving without radar guns, lidar, or calibrated camera angles?**

**Through relative kinematic heuristics in `edge/rash_detector.py`.** We track vehicle bounding boxes using IoU tracking (minimum 5 frames). We flag: (1)

**Speeding:** When a vehicle's pixel displacement velocity exceeds **2.0x the fleet median velocity**; and (2) **Lane Weaving:** When a vehicle's horizontal velocity vector flips sign ( $dx > 0$  to  $dx < 0$ ) **>= 3 times** within its trajectory track. Alerts are backed by kinematic evidence and human review.



Q29: What is the false-positive rate for license plate reading and how do you prevent misreads?

Our false-positive alert rate is near zero due to **3-Frame Temporal Majority Voting**. Even if RapidOCR misreads a single blurry character on frame 1 (e.g., '8' as 'B'), the candidate string is only committed if **at least 3 agreeing reads occur within a 12-second window** along the same spatial tracking key. A 120-second per-plate cooldown prevents spam.

Q30: How does the Smart Route Planner work?

In backend/app/routing.py and SmartRoute.jsx, the routing engine calculates navigational paths using a cost-penalized graph. High-severity incidents (potholes, open manholes) and severe traffic bottlenecks add artificial distance penalties to road segments, automatically directing city fleets and emergency vehicles onto smoother, faster alternative routes.

Category E: Privacy, Legal Compliance & DPDP Act

Q31: You are mounting cameras on public buses and reading license plates. How does this comply with India's DPDP Act 2023?

UrbanLens is built strictly on **Privacy-by-Design and Data Minimization**:

- 1. **Zero Video Retention**: Raw video frames exist solely in volatile RAM during inference (~33ms) and are immediately overwritten. Raw video never touches the SSD or cloud.
- 2. **On-Device Pedestrian Anonymization**: In edge/privacy.py, any detected pedestrian bounding boxes inside evidence crops are pixelated (8x8 downscale + nearest upscale) before JPEG encoding.
- 3. **Data Minimization**: ANPR stores only alphanumeric text, GPS, and timestamp without facial imagery.
- 4. **Purpose Limitation**: Data is accessible only by authorized municipal authorities (PWD, Traffic Police) under strict role-based access control.

Q32: Can a driver or technician steal the SD card and recover private footage of citizens?

**No**. Raw video is never saved to the SD card or persistent storage. The storage holds only the Linux OS, compiled ONNX model binaries, and serialized JSON metadata. Even if the device is stolen, no video recordings exist to be recovered.

Q33: Who owns the road and traffic data collected by the fleet?

The municipal corporation or public transport authority (e.g., DTC, BEST) retains 100% primary data ownership. UrbanLens operates as a technology and analytics platform provider, adhering to municipal data governance policies.

Category F: Business Model, Unit Economics & ROI

Q34: Why would a State Transport Corporation (e.g., DTC, BEST, BMTCL) allow you to install this on their fleet?

**Zero Capex, Zero Driver Burden, and Tangible Fleet Maintenance Savings.**

- The hardware kit installs in 20 minutes with zero modification to bus controls.
- Drivers have zero operational duties.
- Transport corporations receive **free real-time road roughness and pothole heatmaps**, allowing depot managers to optimize route scheduling and reducing bus suspension, tire, and chassis wear-and-tear by an estimated **18% annually**.
- Corporations participate in a revenue-share model on data feeds monetized to road contractors.

Q35: How does UrbanLens compare against existing 311 citizen complaint apps?

Citizen complaint apps are **reactive, biased, and inconsistent**. Over 85% of potholes on secondary streets are never reported because citizens only complain about routes they frequent. Citizen photos lack calibrated GPS, accurate defect classification, and severity scoring. **UrbanLens is proactive and automated**: buses sweep 100% of routes every day, detecting defects the same day they form without relying on citizen effort.

Q36: How does UrbanLens compare against fixed CCTV surveillance poles?

A single smart CCTV pole costs **Rs. 1,00,000 to Rs. 5,00,000** to install, requires continuous high-voltage power, demands dedicated fiber-optic trenching, and covers only a single stationary 50-meter junction. **For the cost of one CCTV pole, UrbanLens equips 10 to 20 buses (at Rs. 22,000 each)**, providing dynamic mobile sensing coverage across hundreds of kilometers of city roads every single day.

Q37: What is your commercial monetization model?

We operate a B2G / B2B SaaS data licensing model:

- 1. **Municipal Infrastructure Subscription**: Annual SaaS license paid by Municipal Corporations (PWD, Smart City SPVs) for the real-time GIS command center, automated work-order dispatch, and contractor audit tools.
- 2. **Road Contractor Verification Feeds**: Monetized data feeds for road construction concessionaires (NHAI contractors) to audit road surface quality before and after repaving.
- 3. **Insurance & Logistics Telematics**: High-resolution road hazard data feeds licensed to commercial freight fleets and automotive insurers for risk modeling.

5. Rapid-Fire Team Flashcards (20 Quick-Drill Questions)

Use these rapid flashcards to quiz each other before facing the judges:

| No. & Question                                              | Definitive Rapid-Fire Answer                                                                                                   |
|-------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| 1. What are the 9 defect classes detected by model.onnx?    | pothole, crack, waterlogging, garbage_dump, illegal_parking, broken_streetlight, open_manhole, roadside_debris, faded_signage. |
| 2. What is the mAP50 of the defect model vs open_manhole?   | Defect model overall mAP50 is 0.669; open_manhole is 0.971.                                                                    |
| 3. What is the plate detector mAP50 and dataset size?       | mAP50 is 0.993; trained on 2,083 Indian license plate images across 40 epochs.                                                 |
| 4. What is the end-to-end detection-to-dashboard latency?   | 1.0 to 2.8 seconds (measured via WebSockets).                                                                                  |
| 5. What are the 3 stages of the edge quality gate filter?   | Class confidence floor (0.55-0.78), 2-frame streak in 1.5s (or single >=0.75), 4.0s per-label cooldown.                        |
| 6. What is the spatial dedup radius and time window?        | 35 meters radius (DEDUP_RADIUS_M = 35.0) and 240 minutes window (DEDUP_WINDOW_MIN = 240).                                      |
| 7. What is the base risk score for open_manhole vs pothole? | open_manhole = 9, pothole = 7 (scale 1 to 10).                                                                                 |
| 8. What is the formula for the Congestion Index?            | congestion_index = min(100, vehicles*12 + speed_penalty + crowd_bump).                                                         |
| 9. How is rash driving speeding detected?                   | When a vehicle's tracked pixel speed exceeds 2.0x the fleet median speed.                                                      |
| 10. How is rash driving lane weaving detected?              | When a vehicle exhibits >= 3 horizontal direction reversals (dx sign flips) along its track.                                   |
| 11. What is the ANPR temporal voting rule?                  | Requires at least 3 agreeing OCR reads within a 12-second window along the same spatial track.                                 |

|                                                                        |                                                                                                                          |
|------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| 12. What regex validates Indian license plates?                        | ^[A-Z]{2}\d{1,2}[A-Z]{0,3}\d{4}\$ (State + RTO + Series + 4 digits).                                                     |
| 13. How are bystanders anonymized on-device?                           | Pedestrian bounding boxes inside evidence crops are pixelated (8x8 downscale + nearest upscale) before JPEG compression. |
| 14. Why is raw video never saved to disk or cloud?                     | Zero video retention by design: frames exist in volatile RAM during inference (~33ms) and are immediately overwritten.   |
| 15. What port does Supabase PgBouncer transaction pooler use?          | Port 6543 (with prepare_threshold=None and NullPool).                                                                    |
| 16. What is the per-bus hardware BOM cost?                             | ~Rs. 22,000 (Raspberry Pi 5 + 1080p camera + GPS + 4G LTE dongle + power regulator + enclosure).                         |
| 17. What external API costs were incurred?                             | Rs. 0 (all free tiers: Supabase, Open-Meteo, OpenAQ, CARTO, Cloudflare).                                                 |
| 18. What is the incident lifecycle state sequence?                     | NEW -> CONFIRMED -> ASSIGNED -> IN_PROGRESS -> RESOLVED.                                                                 |
| 19. How does the dashboard handle React 18 StrictMode double-mounting? | Per-call channel ownership tracking and refreshRef cleanup pattern to prevent duplicate WebSocket subscriptions.         |
| 20. How does the offline fallback work if Supabase is blocked?         | Run start_backend_sqlite.bat (switches to local SQLite) and the React dashboard automatically falls back to 10s polling. |

6. Team Code Map & Quick Reference

When a judge asks 'Show me where X is in the code', reference these exact files and symbols:

| Functionality    | File Path & Module                  | Key Function / Class / Symbol             | Key Constants & Thresholds                                                     |
|------------------|-------------------------------------|-------------------------------------------|--------------------------------------------------------------------------------|
| Edge Agent Loop  | edge/main.py                        | _passes_gates()<br>main()                 | STREAK_NEEDED = 2, STREAK_WINDOW_S = 1.5, COOLDOWN_S = 4.0                     |
| Defect Detection | edge/detector.py                    | Detector<br>load_detector()               | Input 640x640, output (1, 13, 8400), greedy NMS, conf floor 0.55–0.78          |
| Traffic Counting | edge/traffic_counter.py             | TrafficCounter                            | WINDOW_S = 10.0, speed free-flow 40 km/h, crowd bump +20                       |
| ANPR & OCR       | edge/anpr.py                        | ANPR                                      | RapidOCR, Regex ^[A-Z]{2}\d{1,2}[A-Z]{0,3}\d{4}\$, 3-frame vote, 120s cooldown |
| Rash Driving     | edge/rash_detector.py               | RashDetector                              | IoU >= 0.30, min 5 frames, speed > 2.0x median, lane flips >= 3                |
| Privacy Blurring | edge/privacy.py                     | crop_evidence()                           | 8x8 pixel downscale/upscale on pedestrian bounding boxes, JPEG q=70            |
| Spatial Dedup    | backend/app/dedup.py                | find_matching_incident()<br>haversine_m() | DEDUP_RADIUS_M = 35.0m, DEDUP_WINDOW_MIN = 240min, bbox +/- 0.01 deg           |
| Severity Scoring | backend/app/severity.py             | score()                                   | open_manhole: 9, pothole: 7, conf_f: 0.7-1.1, count_f: 1.0-1.5                 |
| DB Pooler Fix    | backend/app/database.py             | get_engine()                              | Supabase port 6543, prepare_threshold=None, poolclass=NullPool                 |
| Realtime WS Sync | dashboard/src/lib/realtime.js       | subscribeToTable()                        | StrictMode cleanup, per-call ownership, 10s polling fallback                   |
| Model Training   | ml/train_v3.py<br>ml/train_plate.py | Ultralytics YOLO train                    | SGD optimizer, lr0=0.01, close_mosaic=10, 40 epochs on RTX 5060                |

7. Live Demo Protocol & Emergency Fail-Safes

DEMO EXECUTION PROTOCOL & FAIL-SAFE PROCEDURES

LIVE DEMO EXECUTION STEP-BY-STEP:

1. **Start Backend:** Run scripts/start\_backend.bat (connects to Supabase on port 6543).
2. **Launch Dashboard:** In dashboard/ run npm run dev -> open http://127.0.0.1:5173.
3. **Launch Edge Agent:** Run scripts/run\_edge.bat (replays demo route with real ONNX inference).
4. **Highlight Live Features:** Point out (a) Marker appearing in 2.5s with blurred crop, (b) Congestion heatmap updates in Traffic tab, (c) ANPR plate detection with watchlist match, (d) Rash driving speed violation flag, and (e) Smart Route re-calculation avoiding defects.

EMERGENCY FAIL-SAFES (IF NETWORK FAILS):

- *If venue WiFi blocks Supabase ports:* Run scripts/start\_backend\_sqlite.bat. The backend instantly switches to local SQLite (urbanlens.sqlite3) and the React dashboard seamlessly switches to its 10-second polling fallback mode.
- *If video device fails:* The edge agent automatically falls back to deterministic recorded video media/route\_demo.mp4.